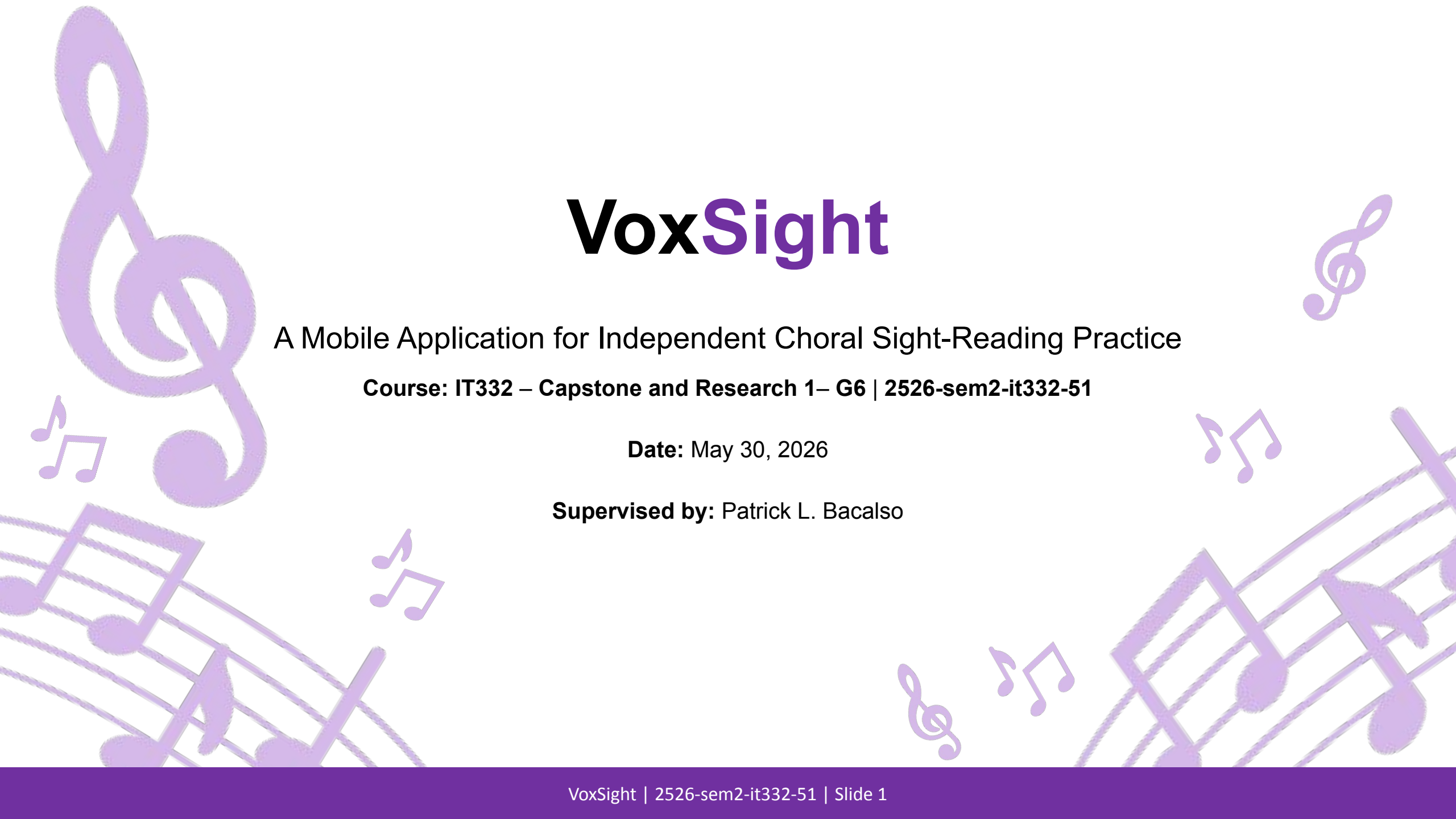




VoxSight

A Mobile Application for Independent Choral Sight-Reading Practice

Course: IT332 – Capstone and Research 1– G6 | 2526-sem2-it332-51

Date: May 30, 2026

Supervised by: Patrick L. Bacalso

The Team



Lagamo Jr., Elvin O.

Project Manager
Backend Developer
QA/Tester



Castillo Jr., Teodoro D.

Frontend Developer
UI.UX
QA/Tester



Sala, Denzel Don L.

Documentation Lead
Frontend Developer
QA/Tester



Sevilla, John Emmanuel S.

Backend Developer
QA/Tester
UI/UX



Velasco, German Oliver E.

Frontend Developer
UI.UX
QA/Tester

Background of the Study / Project Context

The Amateur Choral Dilemma:

Amateur choirs rely heavily on vocalists without formal notation literacy, making them dependent on external training supports (Timoshenko, 2018; Garisi, 2024).

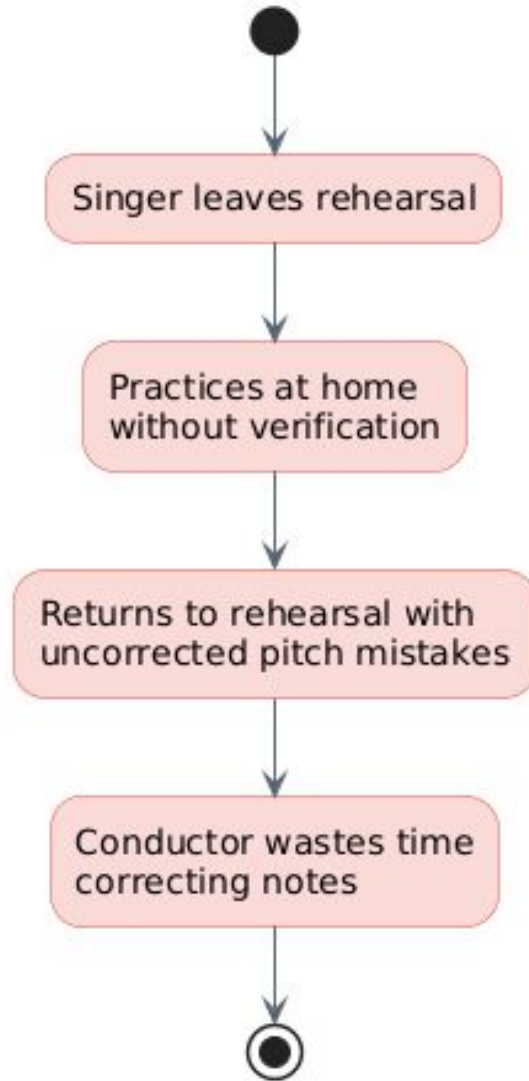
Acoustic Flaws of the Piano:

Traditional rehearsal aids like pianos utilize a percussive attack tone. Singers fundamentally require continuous, sustained harmonic context to achieve accurate vocal tuning (Frampton, 2010).

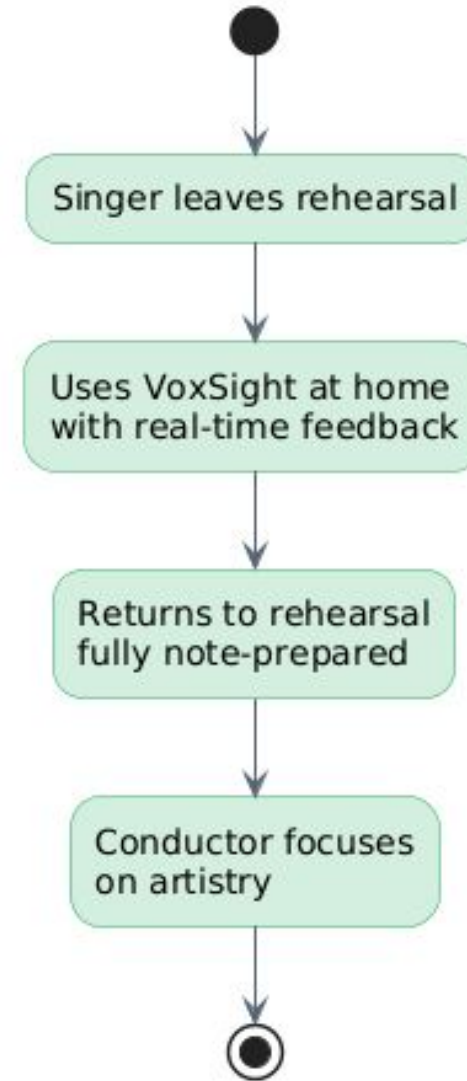
The Rehearsal Bottleneck:

Section-by-section rote teaching ("listen and repeat") leaves up to 75% of the ensemble idle, draining rehearsal time on mechanical note-pounding rather than artistic expression (Adokiye & Jacinta, 2021).

Current Traditional Workflow



VoxSight At-Home Workflow



Problem Statement

Main Problem: Amateur choir members are unable to independently read and verify the accuracy of their vocal parts when practicing repertoire outside of scheduled rehearsals.

Supporting Gaps:

Closed Ecosystems: Existing pitch validation software (e.g., SmartMusic) blocks custom sheet music uploads.

Static Playback: Generic OMR tools provide only flat MIDI piano playback with no vocal tuning support or real-time pitch tracking.

Effects: Slowed down repertoire acquisition, excessive reliance on conductor-led rote instruction, and severe rehearsal pacing bottlenecks.

Stakeholders, Beneficiaries, and Users

Stakeholder	Role	Benefit from the System
Amateur Choir Members	Primary End-Users	Can independently practice custom repertoire with instant pitch verification, minimizing cognitive overload.
Choir Directors / Conductors	Secondary Users / Facilitators	Drastically reduces time spent on raw note-teaching, shifting rehearsals toward artistic and expressive refinement.
Ensemble Administrators	Performance Overseers	Accelerates concert preparation cycles and increases overall musical accuracy across seasonal repertoires.

Proposed Solution Overview

The VoxSight Framework

A client-server mobile companion that integrates computer vision, audio synthesis, and real-time digital signal processing.

Core Engineering Triad



Controlled OMR Digitization

Converts physical scores
into playable, standard
MusicXML data.



Audio-Visual Selective Focus

Dynamically isolates a
user's vocal line while
suppressing background
distractions



Sight-Reading Training

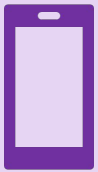
Combines synchronized
playhead tracking with
immediate
microphone-based pitch
analysis

Proposed Solution Overview

The VoxSight Framework

A client-server mobile companion that integrates computer vision, audio synthesis, and real-time digital signal processing.

Technical Stack



Frontend

Android Kotlin, Jetpack
Compose UI, CameraX API,
TarsosDSP Audio Library.



Backend

Java 21, Spring Boot
Framework, Spring Data JPA,
Java ProcessBuilder
Subprocesses.



Data / Engine Layer

Supabase Cloud
Data Storage,
Audiveris.

SMART Goals Overview



Specific

Target individual amateur singers practicing custom 4-part SATB choral arrangements.



Measurable

The system maintains verifiable performance baselines: an OMR parsing time of 10 seconds or less, a playhead sync drift of 0.1 seconds or less, a pitch feedback latency of 0.5 seconds or less, and a frequency tracking accuracy of 85 percent or higher.



Achievable

Run intensive signal processing (TarsosDSP) natively on the device hardware to bypass network round-trip delays, utilizing Spring Boot exclusively for heavy asynchronous orchestration.



Relevant

Directly solves the rehearsal note-pounding bottleneck documented across existing choral literature.



Time-bound

Implemented within a strict Agile Scrum framework consisting of two-week sprint iterations.



General Objective 1

General Objective 1 (Controlled OMR Digitization Module): Automate the conversion of scanned physical sheet music into the playable MusicXML standard with a target structural recognition accuracy of approximately 80–85% under controlled input conditions.

Specific Objectives:

1.1 Structural Part Separation: Separate standard open scores into four distinct, individually addressable SATB vocal tracks.

1.2 Latency Threshold: Process uploaded high-contrast score pages within a maximum processing time of ≤ 10 seconds per page.



General Objective 2

General Objective 2 (Audio-Visual Selective Focus Module): Decrease user cognitive load by dynamically manipulating the digital interface to keep the specific vocal line fully visible while dimming unassigned parts and suppressing background audio interference.

Specific Objectives:

2.1 Visual Staff Dimming: Maintain the user's selected vocal track at 100% visual opacity while dropping non-assigned SATB staves $\leq 20\%$ opacity.

2.2 Auditory Reference Tones: Synthesize human-vowel soundfonts ('Aahs'/'Oohs') instead of percussive pianos, keeping maximum frequency deviations within 10 cents.

2.3 Background Audio Suppression: Actively attenuate non-assigned background vocal lines to near-silent levels during isolated practice.



General Objective 3

General Objective 3 (Sight-Reading Training Module: Dynamic Score Tracking): Emphasize independent sight-reading familiarization by mapping timing data to a visual playhead that highlights active musical notes in real-time.

Specific Objectives:

3.1 Temporal Parsing: Parse and map timestamps and duration values from the MusicXML score into an in-memory queue of NoteEvent array structures.

3.2 Audio-Visual Synchronization: Synchronize visual note-head highlights with the internal audio engine clock, ensuring tracking sync drift stays strictly under ≤ 0.1 seconds (100 ms).



General Objective 4

General Objective 4 (Sight-Reading Training Module: Real-Time Pitch Feedback): Increase user self-correction success within the training environment by utilizing the device microphone to capture vocal input and deliver immediate visual performance tracking.

Specific Objectives:

4.1 Low-Latency Visual Delivery: Display color-coded pitch correctness highlights within ≤ 0.5 seconds of intercepting raw microphone audio data.

4.2 Frequency Detection Reliability: Detect user vocal frequencies with $\geq 85\%$ accuracy against standard digital tuners under quiet environmental conditions.

Scope and Limitations

Scope Boundaries



Platform Target:

Native Android Platforms utilizing local Scoped Storage security features.



Musical Structures: Standard 4-part SATB open scores where each vocal voice type maps to an independent staff line.



Vocal Pitch Ranges Covered: Soprano (C4 to G5), Alto (G3 to D5), Tenor (C3 to G4), and Bass (E2 to C4).

System Limitations



Input Assumptions: Completely excludes handwritten, ornate, or heavily skewed scores.



Formatting Constraints: Does not support condensed short scores (e.g., Soprano and Alto sharing a single treble staff line) or complex divisi arrangements.



Environmental Dependencies: Highly sensitive to noisy ambient environments which can degrade real-time pitch feedback accuracy.



SRS Overview

Purpose

The purpose of this document is to provide a detailed description of the VoxSight system, a mobile application designed to improve independent sight-reading practice efficiency and reduce rehearsal preparation difficulty for amateur choir members.

This document serves as a reference for developers, project managers, and stakeholders to ensure alignment with the functional and non-functional requirements.

Scope

VoxSight is a mobile application targeting the Android platform. The system enables amateur choir members to independently practice their vocal parts on custom sheet music. The system supports standard choral sheet music images (JPEG/PNG/PDF) and four-part SATB voice structures. The system improves independent choir rehearsal preparation through

Four Core Modules:

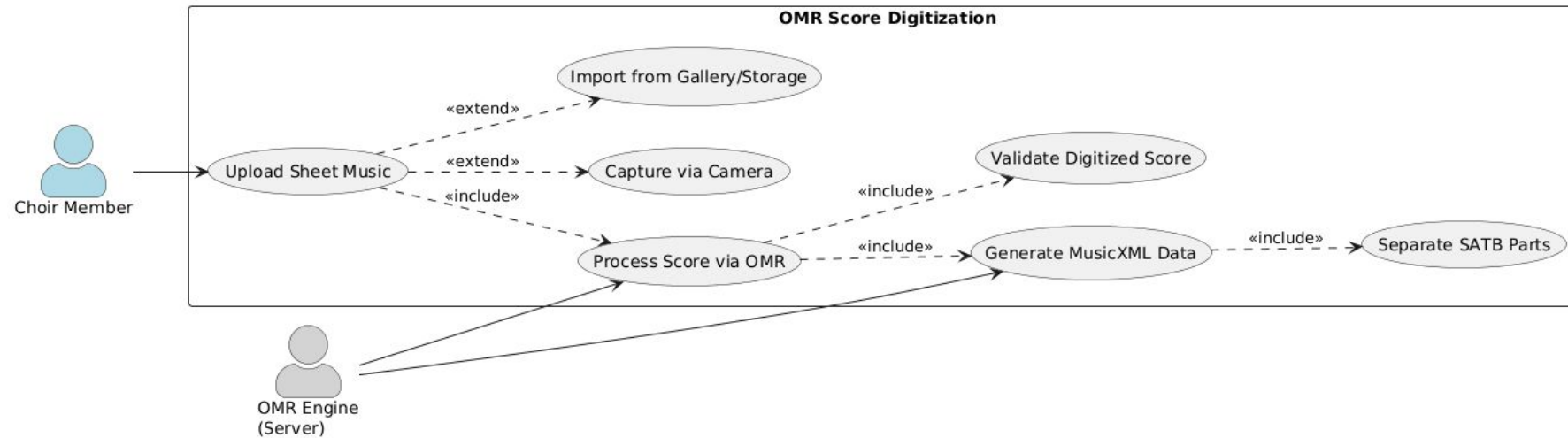
Controlled OMR Digitization, Audio-Visual Selective Focus, Dynamic Score Tracking, and Real-Time Pitch Feedback.

SRS Overview

Core Functional Modules:

Module 1: Controlled OMR Digitization – Secure image capture and automated parsing of physical scores into playable digital music metadata.

Use Case Diagram:

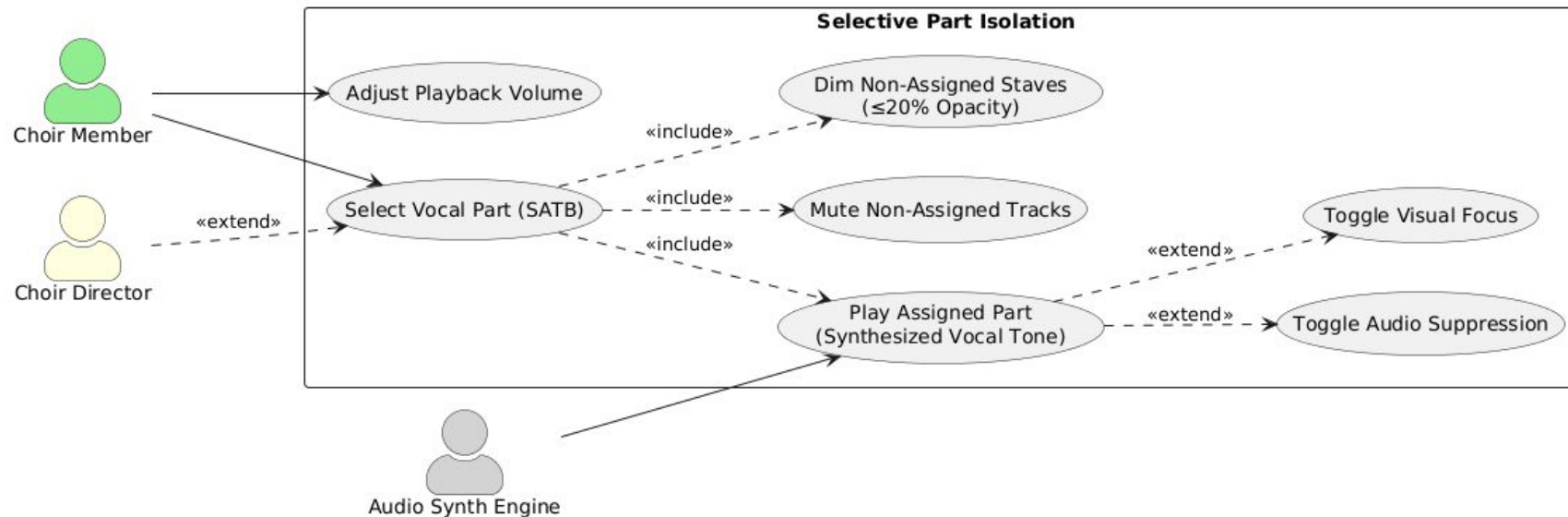


SRS Overview

Core Functional Modules:

Module 2: Audio-Visual Selective Focus – Dynamic synthesis of vocal tracks, allowing individual SATB isolation and background part attenuation $\leq 20\%$ opacity.

Use Case Diagram:

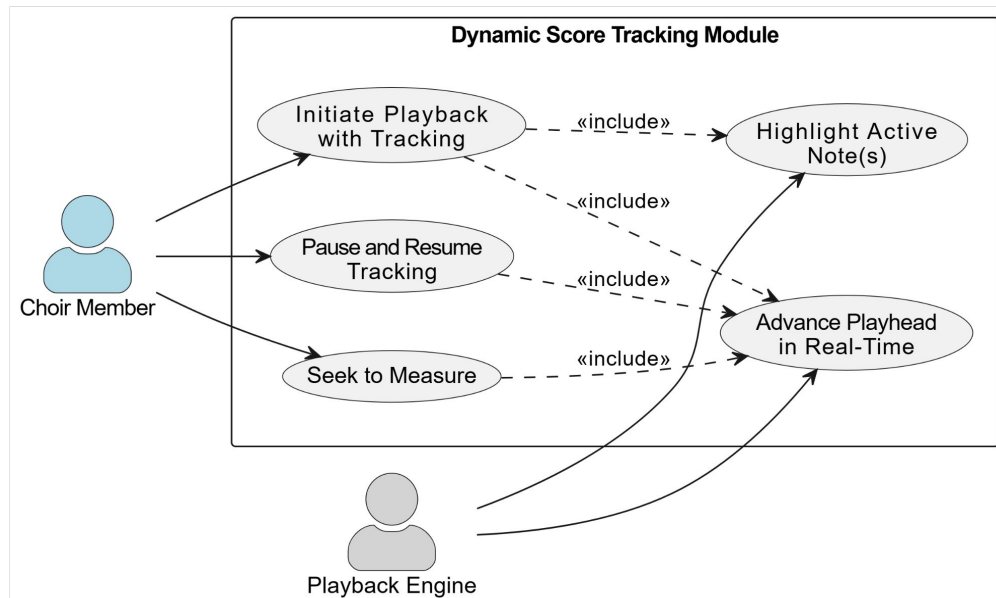


SRS Overview

Core Functional Modules:

Module 3: Dynamic Score Tracking – An animated visual playhead moving across a scrollable canvas, perfectly synchronized with the audio timeline.

Use Case Diagram:

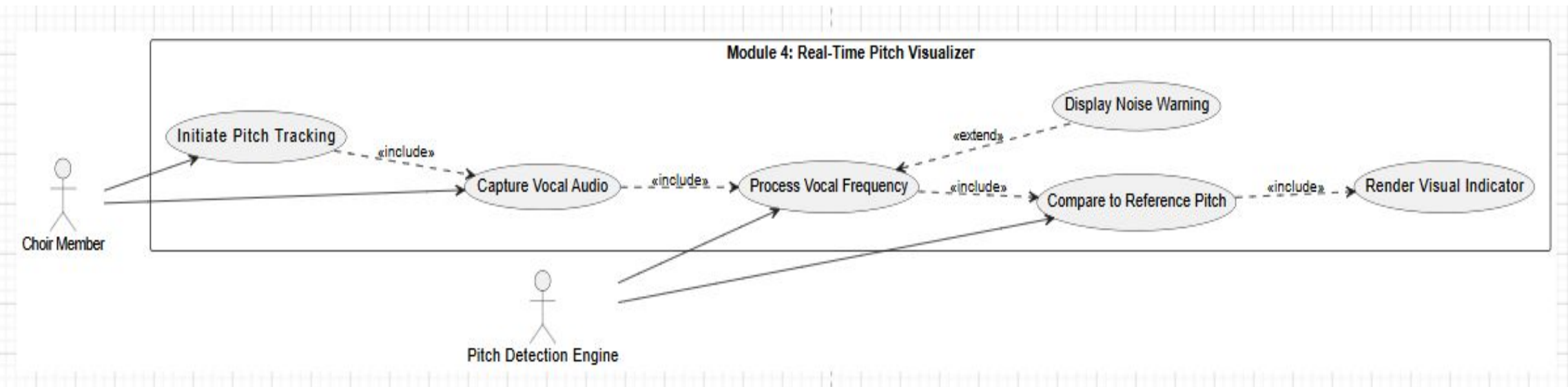


SRS Overview

Core Functional Modules:

Module 4: Real-Time Pitch Feedback – Live microphone audio capture providing instant, color-coded visual accuracy mapping.

Use Case Diagram:





SRS Overview

Non-Functional Performance Baselines:

OMR Speed: ≤ 10 seconds processing turnaround target per standard page.

Sync Latency: Note highlight visual playhead lag capped strictly at ≤ 100 ms relative to audio playback clock.

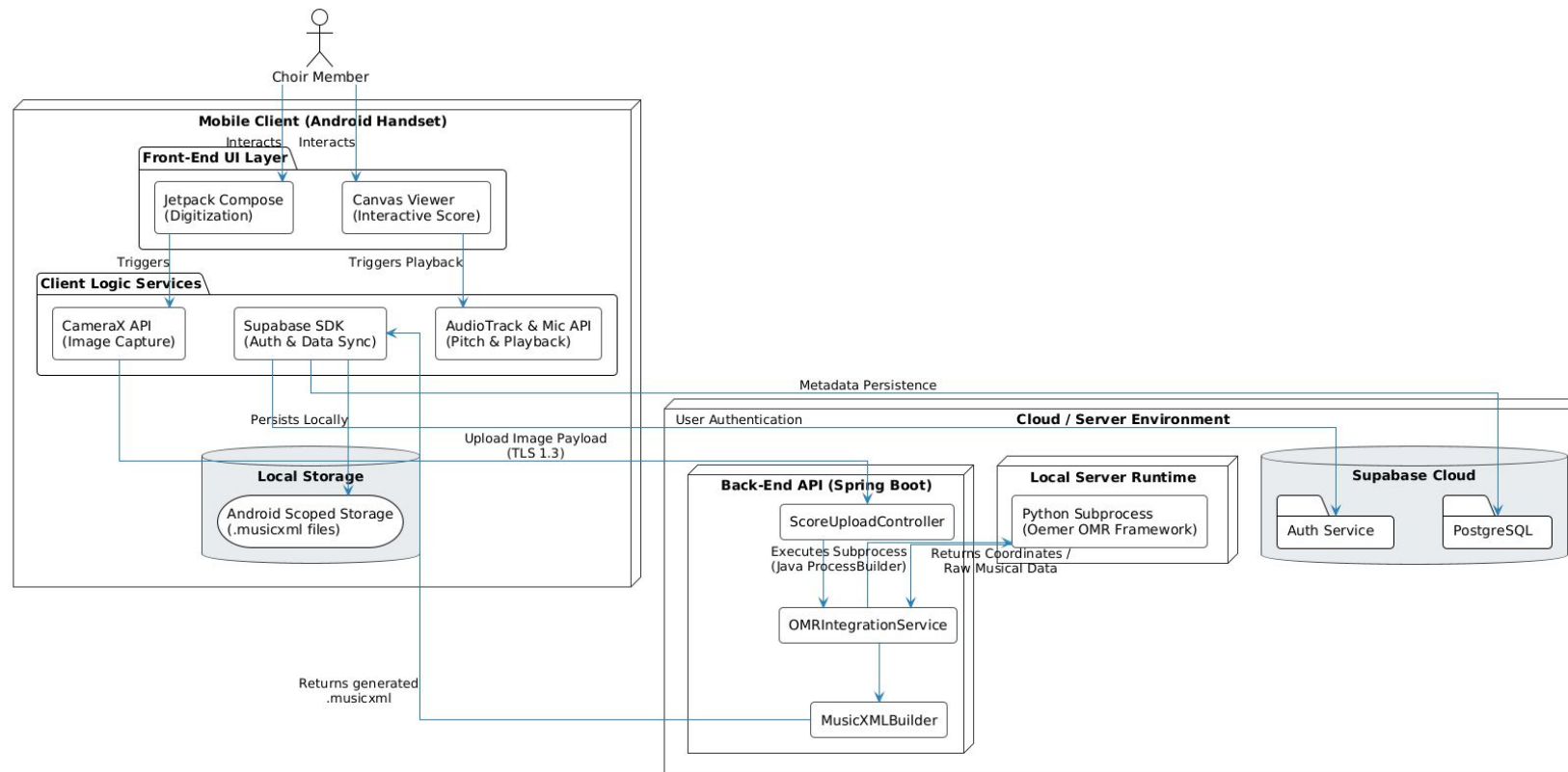
Pitch Bounds: User vocal pitch evaluated dynamically against a standard frequency baseline with a 10 cents reference tone deviation threshold.

Data Security: Mandatory **TLS 1.3** transport encryption for media streams and **Android Scoped Storage** boundaries for files at rest.

SDD Overview

Software Design Document (SDD) — Architecture

VoxSight System Architecture





SDD Overview

Software Design Document (SDD) — Module 1 Details

Module 1: Controlled OMR Digitization Components

Transaction 1.1: Upload Sheet Music

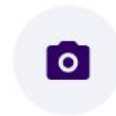
- **Front-End:** UploadScoreScreen (Compose view checking device permissions) & ImageCaptureService (Native Android **CameraX API** verifying resolution).
- **Back-End:** ScoreUploadController (Spring Boot @RestController reading multipart image payloads via secure **TLS 1.3** streams).

🎵 VoxSight



Digitize Score

Import sheet music to generate interactive playback, transpose, and isolate vocal parts.



Take Photo

Scan physical sheets with high precision.



Import File

Upload PDF or image files from your device.



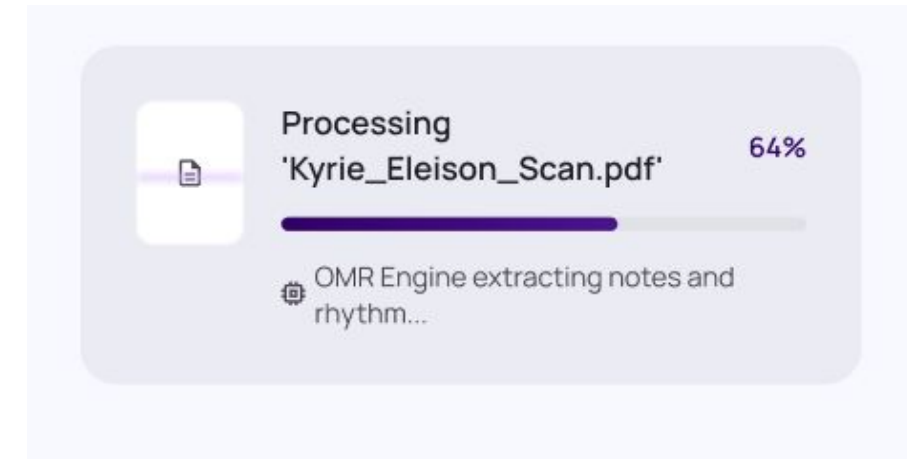
SDD Overview

Software Design Document (SDD) — Module 1 Details

Module 1: Controlled OMR Digitization Components

Transaction 1.2: Process Score via OMR Engine

- **Front-End:** ProcessingLoadingState (Compose screen progress overlay).
- **Back-End:** OMRIntegrationService (Asynchronous @Service spawning a background Python subprocess running the **Oemer OMR Framework** via Java's native **ProcessBuilder** utility class).





SDD Overview

Software Design Document (SDD) — Module 1 Details

Module 1: Controlled OMR Digitization Components

Transaction 1.3: Generate MusicXML & Separate SATB Parts

- **Front-End:** LocalFileManager (Saves downloaded payloads inside **Android Scoped Storage** folders via the Supabase SDK).
- **Back-End:** MusicXMLBuilder (Java Document Object Model DOM component serializing raw Omer pixel positions into a clean, W3C-compliant notation schema).

Recent Scores

[View All](#)

Ave Verum Corpus

Wolfgang Amadeus Mozart

MusicXML

2 days ago



Hallelujah Chorus

George Frideric Handel

MusicXML

1 week ago



O Magnum Mysterium

Morten Lauridsen

MusicXML

2 weeks ago

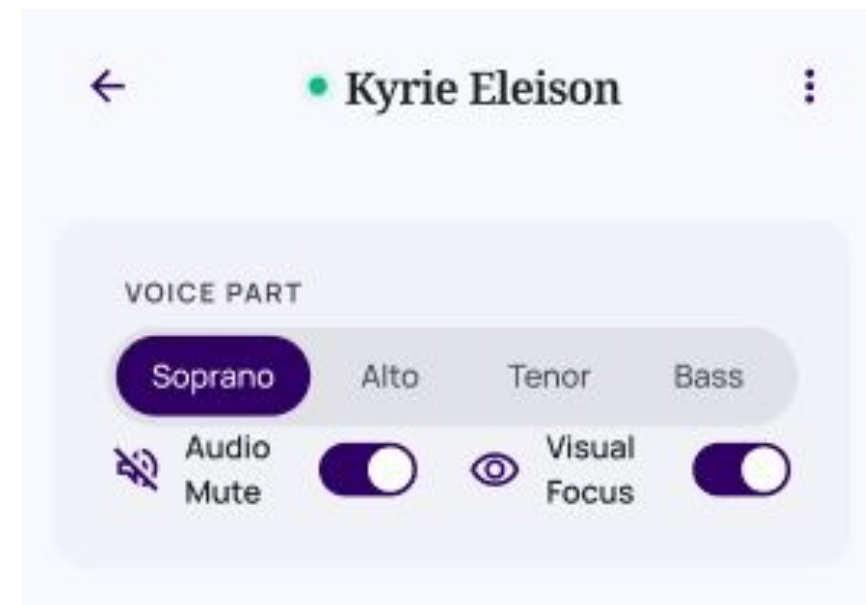
SDD Overview

Software Design Document (SDD) — Module 2 Details

Module 2: Audio-Visual Selective Focus & Part Isolation

Transaction 2.1: Select Voice Part and Apply Focus

- **Front-End:** PartSelectorUI (Jetpack Compose interactive buttons tracking SATB part touch events) & ScoreRenderer (WebView Hybrid) (Android WebView running the JavaScript **Open Sheet Music Display [OSMD]** library; receives focus states via the native VoxSightJsBridge using `updatePartFocus(part, alphaValue)` to execute a 1.0 active vs. 0.2 dimmed alpha-blending loop).
- **Back-End:** AudioVisualMixer (Kotlin ViewModel computing explicit opacity arrays based on selected parts and updating the hybrid bridge layer).





SDD Overview

Software Design Document (SDD) — Module 2 Details

Module 2: Audio-Visual Selective Focus & Part Isolation

Transaction 2.2: Play Assigned Part (Synthesized Vocal Tune)

- **Front-End:** AudioSynthesisEngine (WebView Hybrid) (JavaScript framework running **Tone.js** inside the WebView container; reads sheet metadata via the bridge and schedules note events directly onto the Web Audio API to ensure a maximum frequency deviation of 10 cents & PlaybackController (Kotlin StateFlow utility tracking play, pause, and stop transport frames).
- **Back-End:** LocalScopedStorage (Native Android Scoped Storage API verifying and serving locally preserved score files to bypass internet network reliance).





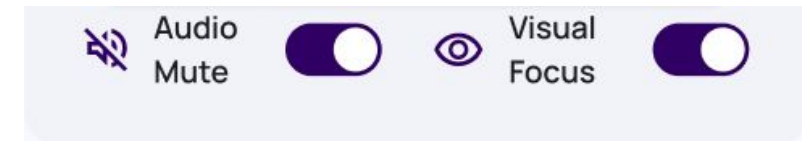
SDD Overview

Software Design Document (SDD) — Module 2 Details

Module 2: Audio-Visual Selective Focus & Part Isolation

Transaction 2.3: Toggle Audio Suppression / Visual Focus

- **Front-End:** FocusToggleSwitch (Compose widget layout that overrides focus logic to force a 100% rendering state) & GlobalStateMonitor (Kotlin StateFlow broadcasting active application-wide isolation modes).
- **Back-End:** UserPreferencesStore (Android Jetpack DataStore for local state cache or remote Supabase DB layer to store persistent settings).



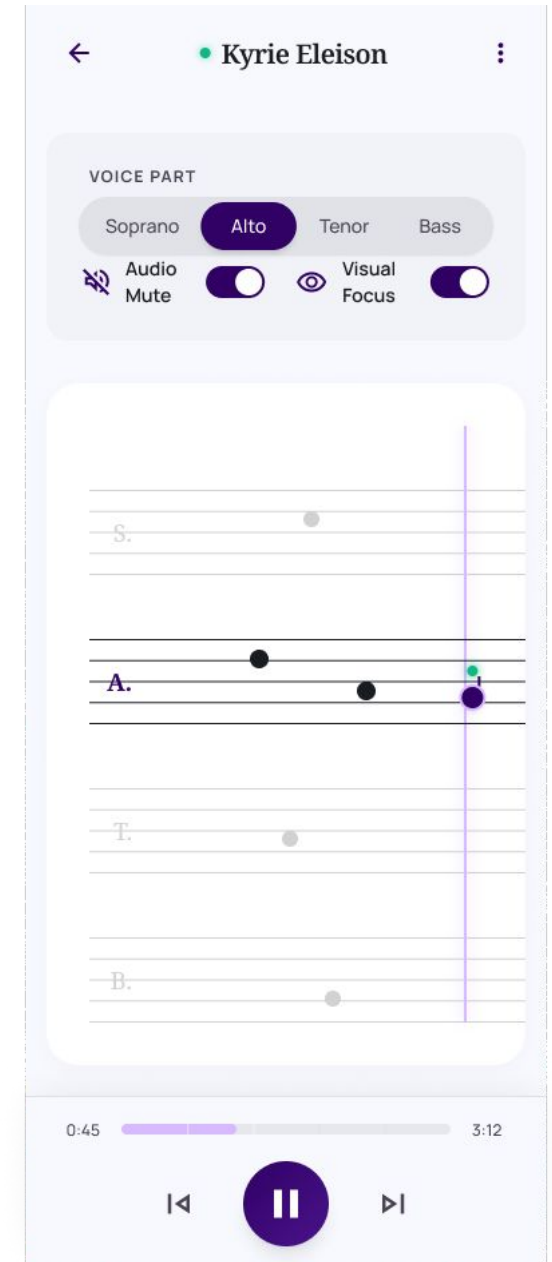
SDD Overview

Software Design Document (SDD) — Module 3 Details

Module 3: Dynamic Score Tracking

Transaction 3.1: Initiate Playback with Tracking

- **Front-End:** ScoreRenderer (WebView Hybrid) (OSMD library rendering sheet music and handling scrolling via JS instructions), PlayheadSynchronizer (Tracks audio clocks to advance the highlight overlay canvas within a strict $\leq 100\text{ ms}$ drift threshold) PlaybackControlBar (Compose transport buttons), & AutoScrollController (Coroutine observer monitoring playhead locations).
- **Back-End:** MusicXMLParser (Kotlin utility parsing files into sequential NoteEvent object vectors) & PlaybackEngineService / SyncDriftMonitor (Background audio tracking service driving an android.media.AudioTrack clock loop; triggers a forced re-sync if the delta exceeds 0.1 seconds).



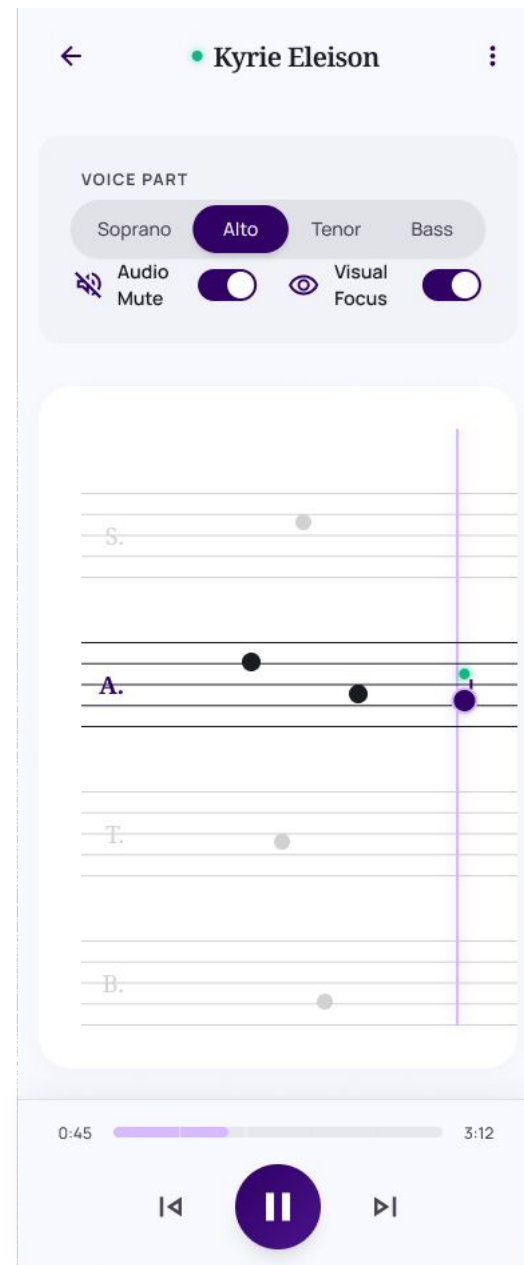
SDD Overview

Software Design Document (SDD) — Module 3 Details

Module 3: Dynamic Score Tracking

Transaction 3.2: Pause and Resume Tracking

- **Front-End:** PlaybackControlBar (Pause/Resume State) (Stateful Compose icon swapper driven by an isPlaying boolean), FrozenPlayheadIndicator (Conditional DrawScope overlay freezing an outlined playhead layout on the score coordinate frame), & SeekTapHandler (Compose pointer touch coordinate input modifier active when paused).
- **Back-End:** PlaybackEngineService (Suspends timing engines and caches positions within pausedAtMs structures) & SeekPositionMapper (Kotlin mapping engine converting raw touch screen pixel coordinates (x, y) against an OSMD-calculated JSON bounding box layout array to seek to the proper startTimeMs stamp).





SDD Overview

Software Design Document (SDD) — Module 3 Details

Module 4: Real-Time Pitch Feedback

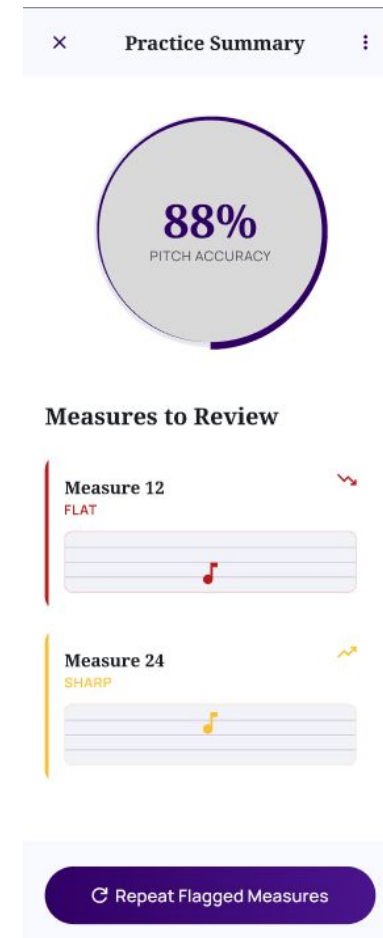
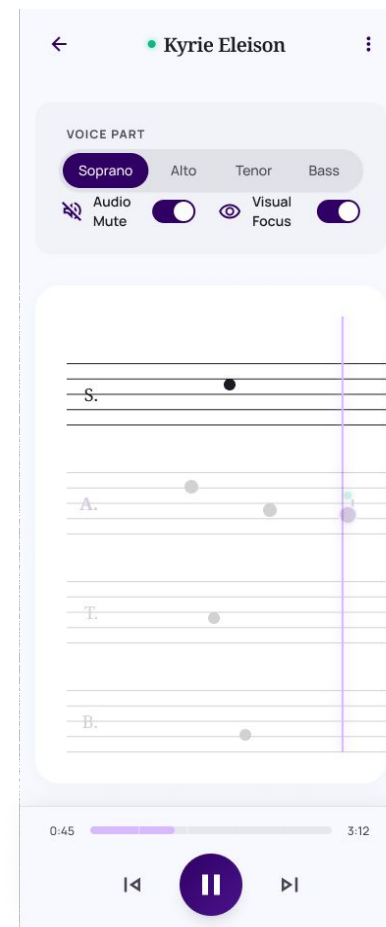
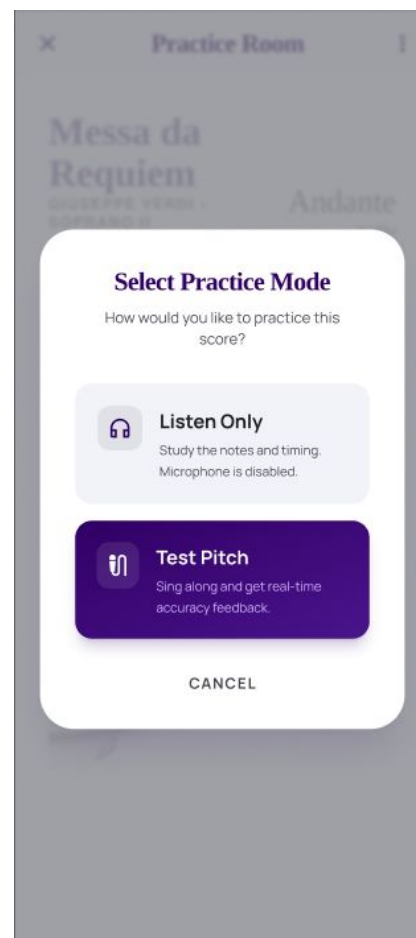
Transaction 4.1: Sight-Reading Training

- **Front-End:** PracticeModePicker (Compose modal view managing state transitions between listening and active microphoned testing), PitchDetectionEngine (Native Kotlin utility reading device raw microphone inputs via the **TarsosDSP framework running the YIN algorithm** with a latency under ≤ 0.5 seconds), VisualFeedbackRenderer (StateFlow UI element calculating pitch deviation in cents: **Green** highlights for inputs within ≤ 10 cents, **Red** for larger deviations)
- **Back-End:** SessionMetricsController (Spring Boot Controller routing secure POST /api/session/save operations to bind incoming JSON payloads directly to a Session DTO object and commit it to database layers via Spring Data JPA repositories).

SDD Overview

Software Design Document (SDD) — Module 3 Details

Module 4: Real-Time Pitch Feedback



Smart Goal Traceability

SMART Goal / Objective	SRS Functional Requirement	SDD Component	Demo Implemented Feature
Digitize custom choir sheets within $\leq 10s$	FR-Upload / FR-Process	ScoreUploadController & OMRIntegrationService	Uploading a high-contrast sheet and seeing the "ProcessingLoadingState" indicator.
Isolate vocal tracks and dim staves to $\leq 20\%$	FR-Focus Selective Layouts	PartSelectorUI & AudioVisualMixer	Selecting "Soprano" to instantly gray out Alto/Tenor/Bass and mute background lines.
Track score positioning under $\leq 0.1s$ drift	FR-Tracking Playhead Sync	PlayheadSynchronizer & PlaybackEngineService	Initiating playback and watching the purple playhead accurately advance note-by-note.
Deliver live pitch color highlights within $\leq 0.5s$	FR-Feedback Real-Time	PitchDetectionEngine & VisualFeedbackRenderer	Singing into the microphone and watching notes light up green or red with target accuracy.

Live Demo: Module 1

Module Overview

Goal: Demonstrate immediate, error-free digitization of a physical or digital choral score sheet and confirm server-side generation of structured music data.

Component Context: UploadScoreScreen + ImageCaptureService -> ScoreUploadController + OMRIntegrationService -> MusicXMLBuilder.

Traceability Chain

SMART GOAL: Digitize custom choir sheets within ≤ 10 Seconds.

SRS REF: FR-Upload / FR-Process (Turnaround processing latency baseline).

SDD REF: ScoreUploadController & OMRIntegrationService (Data Ingestion & Asynchronous Server Execution Layer).

Live Demo: Module 2

Module Overview

Goal: Verify instantaneous isolation of a targeted vocal section (SATB) via coordinated multi-track audio suppression and visual alpha-blended staff dimming.

Component Context: PartSelectorUI -> AudioVisualMixer -> ScoreRenderer (WebView Hybrid) + AudioSynthesisEngine (WebView Hybrid)

Traceability Chain

SMART GOAL: Isolate vocal tracks and dim saves to $\leq 20\%$ Opacity.

SRS REF: FR-Focus Selective Layouts (Vocal track isolation and UI alpha attenuation).

SDD REF: PartSelectorUI & AudioVisualMixer (State Control & JavaScript Hybrid WebView Interface Bridge).

Live Demo: Module 3

Module Overview

Goal: Confirm real-time, fluid advancement of the visual notation tracker in direct synchronization with the active sound engine timeline clock.

Component Context: PlaybackControlBar -> PlaybackEngineService -> PlayheadSynchronizer
-> AuthScrollController -> SeekTapHandler + SeekPositionMapper

Traceability Chain

SMART GOAL: Track score positioning under $\leq 0.1s$ timing drift.

SRS REF: FR-Tracking Playhead Sync (Temporal synchronization limits and playhead cursor stability).

SDD REF: PlayheadSynchronizer & PlaybackEngineService (Low-Latency Native Background Timing Engine & Pointer Coordinate Map Matrix).

Live Demo: Module 4

Module Overview

Goal: Verify native microphone audio analysis and confirm instantaneous, color-coded precision tracking against the reference target score note events.

Component Context: PracticeModePicker -> PitchDetectionEngine -> VisualFeedbackRenderer -> SessionMetricsController.

Traceability Chain

SMART GOAL: Deliver live pitch color highlights within $\leq 0.5s$ of audio input.

SRS REF: FR-Feedback Real-Time (On-device frequency extraction limits and logarithmic matching accuracy).

SDD REF: PitchDetectionEngine & VisualFeedbackRenderer (Digital Signal Processing Infrastructure & Real-Time StateFlow UI Renderer).

Project Accomplishments

Challenges Encountered

Conclusion and Q&A